

Trabajo Práctico Integrador – Metodologías de Sistemas

II Entrega final: repositorio Git + informe + defensa

Fecha:

21/05/2026

Integrantes:

- **Hoffman, Diana**
- **Krumrick, Silvia**
- **Mardon, Florencia**
- **Perrón, Graciela**
- **Sextua, Sandra**

Estructura por semana:

Semana 1 – Refactorización Inicial y Calidad de Código

Objetivo de la semana

Durante la primera semana se realizó una revisión del código base del sistema de feria de emprendedores. El objetivo principal fue identificar problemas de diseño y aplicar una refactorización inicial, manteniendo la misma funcionalidad externa del sistema.

Esta etapa estuvo enfocada en mejorar la claridad del código, reducir duplicaciones, separar responsabilidades, proteger el estado interno de los objetos y aplicar principios básicos de código limpio y SOLID.

1. Code smells detectados

1.1. Nombres poco descriptivos

En la clase Emprendedor se utilizaban atributos con nombres de un solo carácter, como n, t, m, cat y prods. Estos nombres dificultaban la lectura del código y hacían menos clara la intención de cada atributo.

Para corregir este problema, se reemplazaron por nombres descriptivos: nombre, telefono, email, categoria y productos.

1.2. Atributos públicos y falta de encapsulamiento

Se detectó que varias clases exponían directamente sus atributos como public. Esto permitía modificar el estado interno de los objetos desde cualquier parte del sistema, especialmente desde Main y GestorFeria.

Esta situación generaba un diseño poco seguro y difícil de mantener, ya que no existía control sobre cómo se modificaban los datos internos de cada objeto.

1.3. Mezcla de responsabilidades

El método mostrarInfoYValidar() de la clase Emprendedor realizaba dos tareas diferentes: generar información para mostrar y ejecutar reglas de validación de negocio.

Esta mezcla de responsabilidades afectaba la cohesión de la clase y dificultaba la modificación o reutilización del código.

1.4. Código duplicado

En la clase Producto existían dos métodos con lógica similar para determinar si el stock era bajo: hayStockBajo() e isStockBajo().

También se encontró duplicación en la clase Reportes, donde se calculaba de forma redundante el acumulado de ventas totales.

1.5. Clase con demasiadas responsabilidades

La clase GestorFeria concentraba demasiadas tareas: registro de emprendedores, registro de productos, validaciones de stock, cobros, mutación de colecciones y coordinación general del sistema.

Esto generaba un alto acoplamiento y hacía que la clase fuera más difícil

2. Refactorizaciones aplicadas

2.1. Renombrado de atributos

Se reemplazaron los nombres poco claros por nombres descriptivos, mejorando la legibilidad del código y facilitando su comprensión.

- ❖ n → nombre
- ❖ t → telefono
- ❖ m → email
- ❖ cat → categoria
- ❖ prods → productos

2.2. Encapsulamiento de atributos

Los atributos principales de las clases fueron modificados de public a private. De esta forma, el estado interno de los objetos quedó protegido frente a modificaciones directas desde otras clases.

Para acceder a la información o modificarla de manera controlada, se agregaron getters y métodos específicos según la responsabilidad de cada clase.

2.3. Separación de responsabilidades

Se separó la generación de información de la validación de datos. El método mostrarInfoYValidar() fue reemplazado por una estructura más clara, donde la salida de datos queda separada de las reglas de validación.

La generación de información quedó a cargo de un método específico, mientras que las validaciones fueron delegadas a la clase Validadores.

2.4. Centralización de validaciones

Se reutilizó la clase Validadores para concentrar validaciones de email, teléfono, categoría, precio y stock.

Esto permitió evitar reglas dispersas en distintas clases y dejó el sistema más preparado para incorporar nuevas validaciones en el futuro con menor impacto sobre el resto del código.

2.5. Eliminación de lógica duplicada

En la clase Producto se eliminó la duplicación relacionada con la consulta de stock bajo. La lógica quedó unificada en un único método llamado `tieneStockBajo()`.

En la clase Reportes se reutilizó el método `calcularVentasTotales()` dentro del resumen ejecutivo, evitando repetir el mismo cálculo en distintos lugares.

2.6. Protección de listas internas

Se evitó el acceso directo a las listas internas desde Main y Reportes. Para ello, se agregaron métodos públicos en `GestorFeria` que permiten registrar emprendedores, registrar productos, registrar ventas y consultar información sin exponer directamente la estructura interna de datos.

Con esta modificación, las colecciones internas quedan protegidas y el sistema mantiene un mayor control sobre las operaciones que se realizan sobre ellas.

3. Principios SOLID aplicados

3.1. Principio de Responsabilidad Única (SRP)

Se aplicó el principio de responsabilidad única al separar mejor las funciones de cada clase.

- ❖ Emprendedor representa los datos y comportamientos propios de un emprendedor.
- ❖ Producto representa los datos y comportamientos propios de un producto.
- ❖ Validadores concentra las reglas de validación.
- ❖ `GestorFeria` coordina las operaciones generales de la feria.
- ❖ Reportes se encarga de generar información resumida del sistema.

Gracias a esta separación, cada clase queda más enfocada en una responsabilidad concreta, lo que mejora la claridad y mantenibilidad del sistema.

3.2. Principio de Abierto/Cerrado (OCP)

La refactorización dejó el código más preparado para extender funcionalidades sin modificar directamente toda la lógica existente.

Por ejemplo, al centralizar validaciones y separar responsabilidades, futuras reglas de negocio podrán agregarse con menor impacto sobre las clases principales.

3.3. Encapsulamiento como mejora de diseño

Aunque el encapsulamiento no es un principio SOLID en sí mismo, fue una mejora importante aplicada durante esta etapa.

Al proteger los atributos y listas internas, se evita que otras clases modifiquen directamente el estado de los objetos. Esto permite mantener mayor control sobre los datos y reduce errores derivados de accesos indebidos.

Semana 2 – Patrón de Diseño

4. Problema identificado

Durante la revisión arquitectónica del sistema, se detectó un problema estructural crítico centralizado en la clase GestorFeria. A pesar de las refactorizaciones iniciales, esta clase continuaba operando bajo el *code smell* conocido como "Clase Dios". GestorFeria acumulaba múltiples responsabilidades de negocio: la validación y registro de emprendedores, el control de inventario y stock de los productos, y la lógica de facturación y cobro de ventas. Este alto nivel de acoplamiento provocaba que cualquier cambio en la lógica de un módulo específico impactara directamente en toda la clase coordinadora, violando los principios de alta cohesión y bajo acoplamiento.

5. Patrón de diseño implementado

Para resolver esta deficiencia arquitectónica, se optó por implementar el patrón de diseño estructural Facade (Fachada).

Se procedió a dividir y encapsular la lógica interna de la feria creando un subsistema compuesto por tres clases especialistas:

- ❖ GestorEmprendedores: Administra la colección de usuarios y sus validaciones.
- ❖ GestorInventario: Centraliza la creación, búsqueda y modificación de stock de artículos.
- ❖ GestorVentas: Controla el registro de transacciones comerciales y procesa los pagos.

Simultáneamente, la clase original GestorFeria fue rediseñada para actuar puramente como la "Fachada". Ahora, carece de lógica de negocio propia y de colecciones directas; en su lugar, mantiene referencias a los objetos del subsistema y actúa como un punto de acceso unificado, delegando las peticiones del cliente (la clase Main) al componente interno correspondiente.

6. Mejora obtenida:

La implementación de Facade permitió dismantelar la "Clase Dios" logrando un cumplimiento estricto del Principio de Responsabilidad Única (SRP). El cliente (Main) y los servicios externos (Reportes) continúan interactuando con una interfaz sencilla e intacta. Asimismo, si a futuro se requiere modificar el motor de almacenamiento de productos (por ejemplo, conectándolo a una base de datos), el cambio quedará aislado únicamente dentro de GestorInventario, respetando el principio de Abierto/Cerrado (OCP).

Diagrama UML aplicando Patrón Facade.

